

Partie 1 TP N°2

Gestion des interruptions externes

Bibliothèques

Durée max conseillée : 3h



Objectif général

Passer d'un programme cyclique simple à une application embarquée réactive, structurée autour des interruptions, de bibliothèques logicielles et de capteurs.

Partie lecture durée conseillée : 5 min

1 Préambule.

1.1 Objectifs pédagogiques

- Comprendre le principe d'une interruption matérielle et la différence avec une scrutation dans loop().
- Mettre en œuvre une interruption externe avec un bouton poussoir.
- Utiliser un encodeur incrémental pour détecter un déplacement et son sens de rotation.
- Comprendre le rôle d'une bibliothèque Arduino et son intérêt pour l'abstraction matérielle.
- Installer une bibliothèque depuis le gestionnaire de bibliothèques de l'IDE Arduino.
- Identifier et exploiter les programmes d'exemple fournis avec une bibliothèque.
- Piloter un afficheur Grove LCD RGB à l'aide de sa bibliothèque.
- Acquérir des températures avec un DS18B20 et une température/humidité avec un DHT11.
- Structurer des mesures sous la forme d'un message JSON imposé par un cahier des charges.
- Concevoir une interface de menus utilisant un encodeur incrémental pour naviguer entre plusieurs informations.

1.2 Prérequis

Les connaissances développées dans le TP1 sont considérées comme acquises.

- Utiliser l'IDE Arduino : créer, compiler et téléverser un programme.
- Configurer une entrée ou une sortie numérique avec pinMode().
- Lire une entrée numérique avec digitalRead().
- Utiliser setup() et loop().
- Utiliser le Moniteur Série avec Serial.begin(), Serial.print() et Serial.println().
- Comprendre les principes élémentaires du C/C++ : variables, fonctions, conditions et boucles.
- Identifier les principales broches d'une Arduino Uno.

1.3 Introduction

Dans le TP1, le programme exécutait principalement une succession d'instructions dans loop(). Cette organisation convient aux applications simples, mais elle devient moins adaptée lorsqu'un système doit réagir rapidement à un événement externe, par exemple l'appui sur un bouton ou le déplacement d'un encodeur.

Ce TP introduit donc une évolution importante de la programmation embarquée : le système devient réactif. Une interruption permet au microcontrôleur de suspendre momentanément l'exécution normale du programme afin d'exécuter une fonction associée à un événement matériel.

Le TP introduit également la notion de bibliothèque. Une bibliothèque regroupe du code réutilisable qui permet d'exploiter un composant sans devoir développer soi-même toute la couche logicielle de bas niveau. L'étudiant devra apprendre à rechercher une bibliothèque, l'installer, consulter ses exemples et en extraire les éléments utiles au cahier des charges.

Organisation du TP

Partie	Activité	Approche
	Interruption avec bouton	Guidée
	Encodeur incrémental	Guidée puis application
	Découverte des bibliothèques + LCD RGB	Guidée
	DS18B20 + DHT11 + JSON	Autonomie progressive
	Projet de synthèse : menus	Autonomie

2 Matériel nécessaire

2.1 Carte de développement

La plateforme utilisée est la même que dans le TP1 : Arduino Uno équipée d'une carte d'interface Grove. Le microcontrôleur utilisé est l'ATmega328P.

Élément	Rôle
Arduino Uno	Microcontrôleur et plateforme de développement
Grove Base Shield / carte d'interface	Connexion simplifiée des modules Grove
Bouton poussoir Grove	Génération d'un événement d'entrée
Encodeur incrémental Grove	Détection d'une rotation et de son sens
LCD RGB Grove	Affichage de textes et de valeurs
DS18B20	Mesure de température
DHT11	Mesure de température et d'humidité
Câble USB	Alimentation, programmation et communication série
Ordinateur	IDE Arduino et documentation

2.2 Broches utilisées

Le câblage Grove peut être adapté au matériel disponible. Une contrainte est imposée : les deux entrées d'interruption externes de l'Arduino Uno sont réservées au bouton et à l'encodeur.

Fonction	Broche proposée	Remarque
Bouton poussoir	D4	Lecture état
Encodeur voie A	D2	INT0 — interruption externe imposée
Encodeur voie B	D3	INT1 — interruption externe imposée
LCD RGB	I ² C : SDA/SCL	Connexion Grove/I ² C
DS18B20	D5	Broche numérique à adapter si nécessaire
DHT11	D6	Broche numérique à adapter si nécessaire

Avant toute modification du câblage, débrancher l'alimentation USB. Vérifier l'orientation des connecteurs Grove et demander une validation en cas de doute.

Le câblage présenté dans ce document constitue une proposition de travail et peut être adapté par l'enseignant.

Partie activités durée conseillée : 30 min

3 Programmation événementielle : les interruptions

3.1 Pourquoi utiliser une interruption ?

Avec une scrutation classique, le programme exécute périodiquement `digitalRead()` afin de déterminer si une entrée a changé d'état. L'événement n'est donc détecté que lorsque le programme atteint l'instruction de lecture.

Une interruption permet au microcontrôleur de réagir à un événement sans attendre que `loop()` arrive à une instruction de lecture de l'entrée. Sur l'Arduino Uno, les broches D2 et D3 sont associées aux interruptions externes `INT0` et `INT1`.

3.2 Principe général

Élément	Rôle
Événement	Changement d'état sur une entrée
Interruption	Mécanisme matériel qui signale l'événement au CPU
ISR	Interrupt Service Routine : fonction exécutée lors de l'interruption
<code>attachInterrupt()</code>	Associe une interruption à une fonction
<code>digitalPinToInterrupt()</code>	Permet d'associer une broche à son numéro d'interruption
<code>volatile</code>	Indique qu'une variable peut être modifiée hors du flot principal

Objectif : mettre en œuvre une interruption externe afin de détecter l'appui sur un bouton et commander une LED en réaction à cet événement, en utilisant une routine de service d'interruption et une variable `volatile`.

À l'issue de cette activité, l'étudiant sera capable de :

1. **Identifier le principe d'une interruption**
 - Distinguer une interruption d'une scrutation (*polling*) ;
 - Expliquer le rôle d'une interruption dans un système embarqué.
2. **Configurer une interruption externe**
 - Identifier une broche compatible avec une interruption externe ;
 - Utiliser `digitalPinToInterrupt()` ;
 - Utiliser `attachInterrupt()` ;
 - Choisir le mode de déclenchement (`FALLING`, `RISING`, `CHANGE`, `LOW`).
3. **Écrire une routine de service d'interruption (ISR)**
 - Déclarer une fonction d'interruption ;
 - De comprendre le rôle d'une ISR ;
 - Limiter les opérations réalisées dans une ISR.
4. **Échanger une information entre l'ISR et le programme principal**
 - Utiliser une variable volatile ;
 - Comprendre pourquoi cette déclaration est nécessaire ;
 - Utiliser un drapeau événementiel (`eventButton`) pour signaler l'événement au programme principal.

5. Traiter un événement dans la boucle principale

- Détecter l'événement dans loop() ;
- Acquitter/réinitialiser le drapeau ;
- Réaliser l'action associée à l'événement.

3.3 Fonction attachInterrupt()

3.3.1 Programme

La fonction attachInterrupt() permet d'associer une **routine de service d'interruption (ISR)** à une interruption externe.

attachInterrupt(digitalPinToInterrupt(broche),
nom_ISR,
mode);

- **broche** : numéro de la broche numérique utilisée pour déclencher l'interruption.
- **nom_ISR** : nom de la routine de service d'interruption appelée lorsque l'interruption se produit.
- **mode** : condition de déclenchement de l'interruption :
 - RISING : changement de LOW vers HIGH
 - FALLING : changement de HIGH vers LOW
 - CHANGE : tout changement d'état
 - LOW : niveau logique LOW

Exemple :

digitalPinToInterrupt(2), buttonISR, FALLING);

Ici, la routine buttonISR() est exécutée lorsqu'un **front descendant** est détecté sur la broche numérique D2.

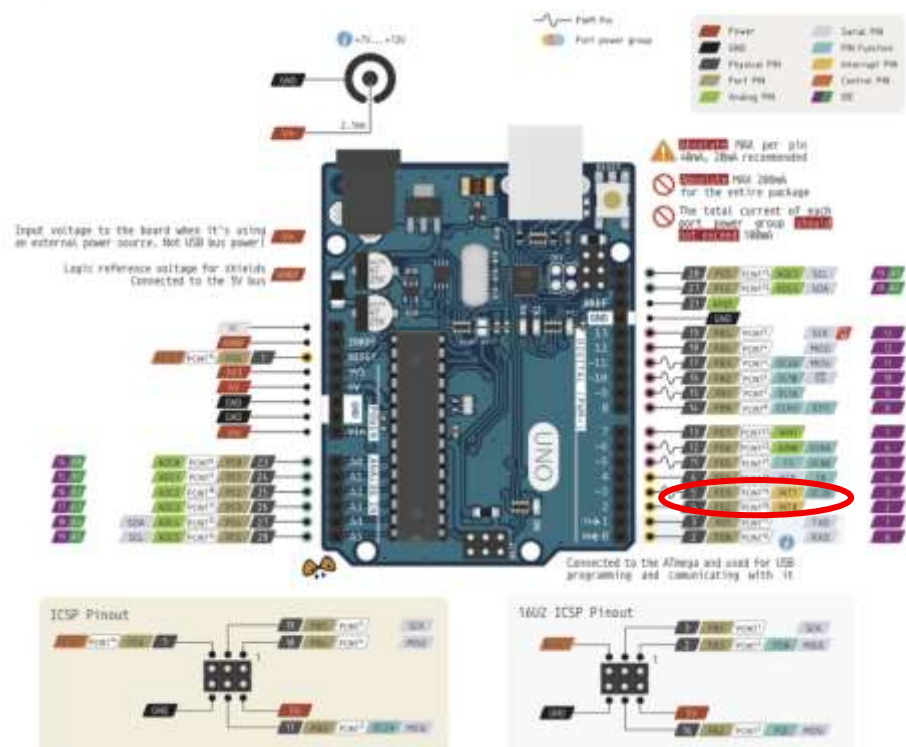
3.3.2 Matériel

Broches utilisables pour les interruptions externes

Sur une **Arduino Uno** basée sur l'**ATmega328P**, les interruptions externes peuvent être utilisées sur les broches numériques :

- D2 → interruption INT0
- D3 → interruption INT1

La fonction digitalPinToInterrupt() permet d'associer automatiquement le numéro de la broche à l'interruption correspondante.

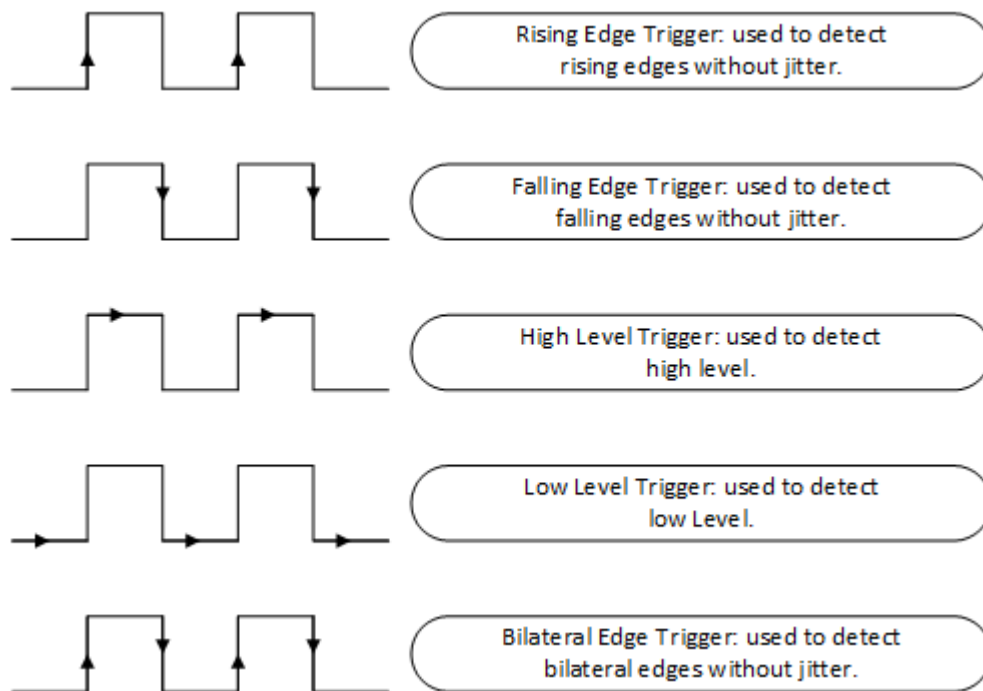


À retenir : sur l'Arduino Uno, les interruptions externes utilisées avec `attachInterrupt()` sont disponibles sur les broches **D2 et D3**.

3.3.3 Modes de déclenchement d'une interruption

Une interruption peut être déclenchée selon différentes **conditions appliquées au signal d'entrée**. Le déclenchement peut avoir lieu lors d'un **changement d'état (front)** ou lorsqu'un **niveau logique particulier** est détecté.

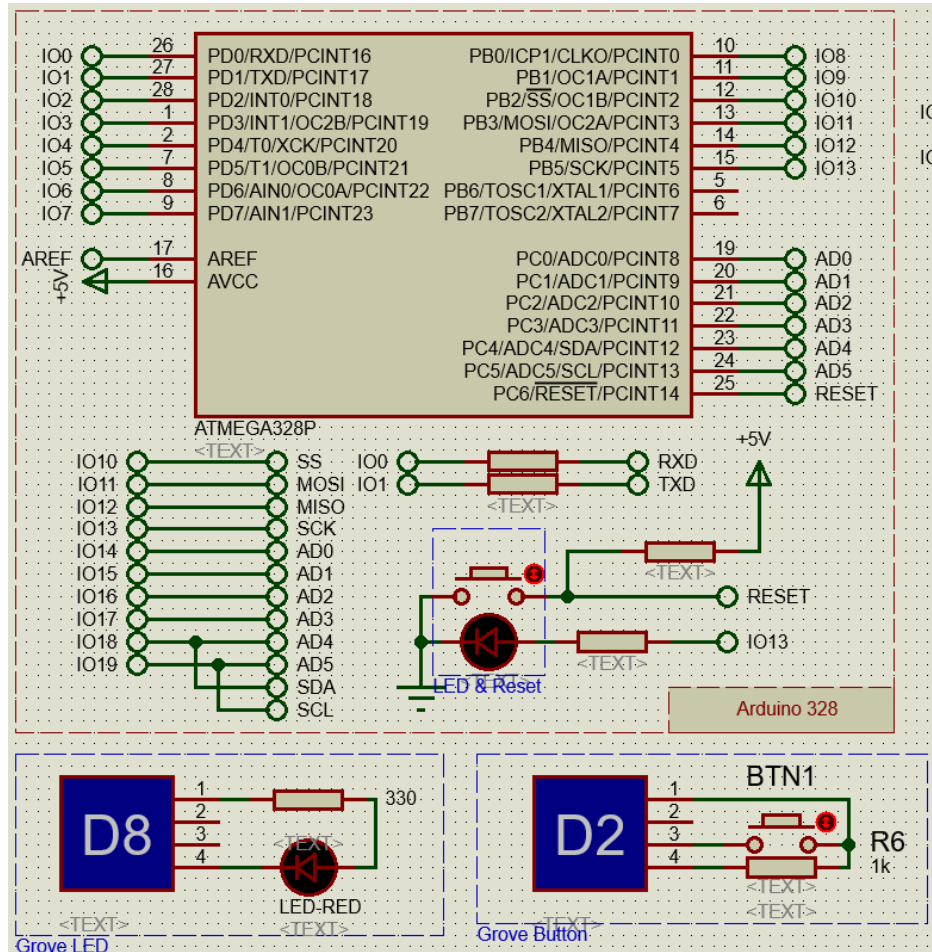
L'illustration présente les principaux modes de déclenchement : **front montant, front descendant, niveau haut, niveau bas et changement d'état**.



3.4 Travail pratique n°1 – Bouton sur interruption

Le bouton est connecté sur D2. La LED de test est connectée sur D8. Chaque appui doit provoquer une action détectée par interruption.

Schéma structurel



Programme à compléter

```
// Bouton-poussoir avec interruption externe
// Broche du bouton-poussoir
// Sur Arduino UNO, les interruptions externes sont disponibles
// notamment sur les broches 2 et 3.
const int buttonPin = ____;
// Broche de la LED
const int ledPin = ____;
// Drapeau indiquant qu'un appui sur le bouton a été détecté.
// volatile est nécessaire car cette variable est modifiée
// dans une routine d'interruption.
volatile bool eventButton = ____;
//
// Routine d'interruption (ISR)
// Cette fonction est exécutée automatiquement lorsqu'un
// événement est détecté sur la broche du bouton.
```

```
void buttonISR()
{
    // Signaler qu'un événement bouton a été détecté.
    eventButton = ____;
}
void setup()
{
    // Le bouton utilise la résistance de rappel interne.
    pinMode(buttonPin, _____);
    // La LED est une sortie.
    pinMode(ledPin, _____);
    // Associer l'interruption de la broche du bouton
    // à la fonction buttonISR().
    // digitalPinToInterrupt() permet d'obtenir le numéro
    // d'interruption correspondant à la broche.
    //
    // FALLING signifie que l'interruption est déclenchée
    // lors d'un changement de HIGH vers LOW.
    attachInterrupt(
        digitalPinToInterrupt(buttonPin),
        _____,
        _____
    );
}
void loop()
{
    // Vérifier si une interruption a signalé
    // un appui sur le bouton.
    if (_____)
    {
        // Réinitialiser le drapeau avant de traiter
        // l'événement.
        eventButton = ____;
        // Inverser l'état actuel de la LED.
        // Si elle est éteinte → elle s'allume.
        // Si elle est allumée → elle s'éteint.
        digitalWrite(
            ledPin,
            !digitalRead(ledPin)
        );
    }
}
```

Travail demandé

- 1- Réaliser le câblage du bouton sur D2 et de la LED sur D8.
- 2- Identifier le mode de fonctionnement de l'entrée INPUT_PULLUP.
- 3- Identifier le rôle de attachInterrupt().
- 4- Identifier le rôle de digitalPinToInterrupt().
- 5- Expliquer pourquoi la fonction buttonISR() est très courte.
- 6- Expliquer pourquoi eventButton est déclarée volatile.

- 7- Télverser le programme et vérifier que chaque appui est détecté.
- 8- Modifier le programme afin d'incrémenter un compteur à chaque événement.
- 9- Afficher le compteur dans le Moniteur Série.

Bonnes pratiques dans une routine d'interruption

- Une ISR doit être aussi courte que possible.
- Éviter les fonctions lentes dans une ISR, notamment les affichages série.
- Éviter delay() dans une ISR.
- Utiliser des variables partagées avec prudence.
- Le traitement complexe peut être réalisé dans loop() après positionnement d'un indicateur d'événement.

Questionnaire

1. Quelle différence existe-t-il entre scrutation et interruption ?
2. Pourquoi D2 est-elle adaptée à l'interruption externe sur Arduino Uno ?
3. Quel est le rôle de attachInterrupt() ?
4. Quel est le rôle de FALLING ? Citer deux autres modes possibles.
5. Pourquoi une variable modifiée dans une ISR doit-elle souvent être déclarée volatile ?
6. Pourquoi faut-il éviter Serial.println() dans une ISR ?

À vous de jouer !

Programmez le système afin de compter le nombre de personnes présentes dans la salle.

Lorsqu'une personne franchit la porte :

- si le capteur **A est activé avant B**, le nombre de personnes présentes est **incrémenté** ;
- si le capteur **B est activé avant A**, le nombre de personnes présentes est **décrémenté**.

Utilisez une **interruption** pour détecter le passage devant le capteur A et déterminez le sens du déplacement à partir de l'état du capteur B.

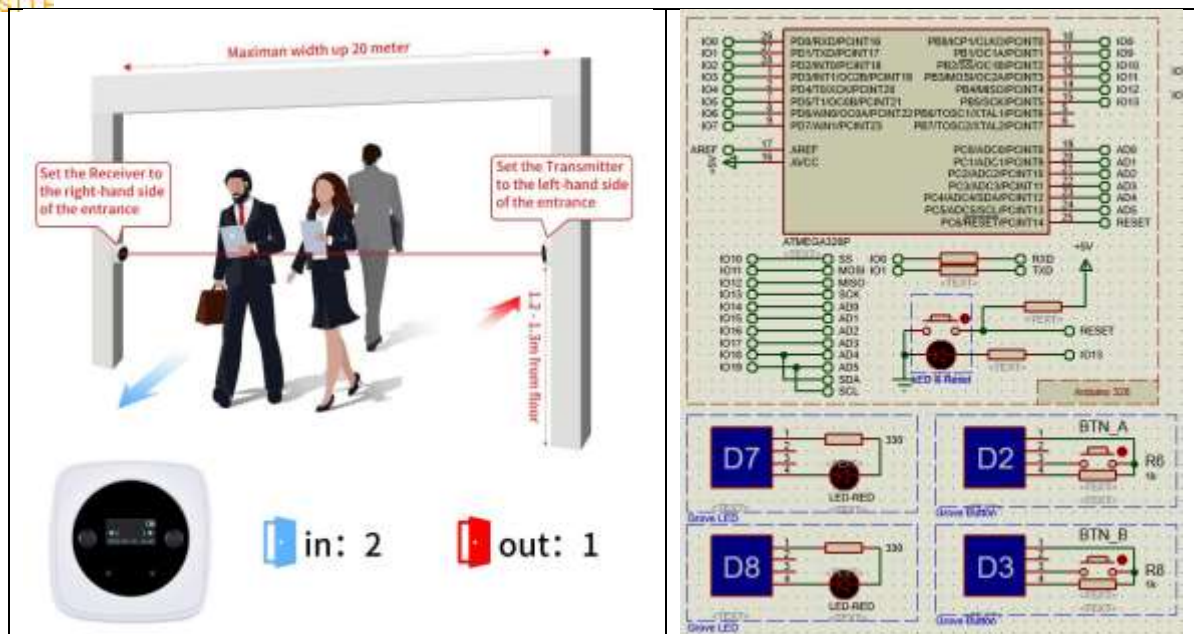
Le nombre de personnes présentes dans la salle sera affiché sur le **Moniteur Série**.

Pour une fiche pédagogique, je trouve même intéressant de faire apparaître la problématique juste avant l'API :

Problématique :

Comment permettre au microcontrôleur de détecter rapidement le passage d'une personne et de déterminer automatiquement si elle entre ou sort de la salle ?

Principe de fonctionnement	Maquette à câbler
----------------------------	-------------------



Partie activités durée conseillée : 30 min

4 Encodeur incrémental et interruptions

Objectif : Mettre en œuvre un codeur incrémental afin de détecter une rotation, déterminer son sens et compter les impulsions à l'aide d'une interruption matérielle sur Arduino.



À l'issue de cette activité, l'étudiant sera capable de :

- Identifier le principe de fonctionnement d'un codeur incrémental et le rôle des voies A et B.
- Expliquer le déphasage entre les signaux A et B permettant de déterminer le sens de rotation.
- Utiliser une interruption matérielle pour détecter les changements d'état d'un signal provenant du codeur.
- Configurer une broche d'interruption avec `attachInterrupt()` et `digitalPinToInterrupt()`.
- Écrire une routine de service d'interruption (ISR) permettant de traiter les impulsions du codeur.
- Déterminer le sens de rotation à partir de l'état des voies A et B.
- Compter les impulsions afin de déterminer une position relative du codeur.
- Utiliser le mot-clé volatile pour déclarer une variable modifiée dans une routine d'interruption.
- Transmettre la position du codeur sur le moniteur série afin de vérifier le fonctionnement du programme.

- Identifier les perturbations liées aux rebonds des contacts et comprendre la nécessité d'un traitement anti-rebond.

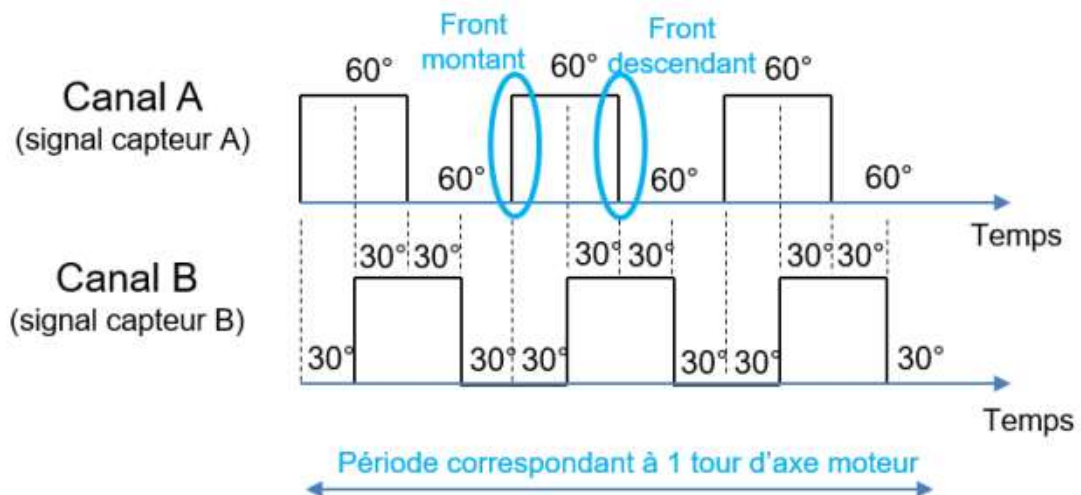
4.1 Principe

Un encodeur incrémental fournit deux signaux numériques déphasés, généralement appelés A et B. L'ordre des transitions permet de déterminer le sens de rotation. Le nombre de transitions permet d'estimer un déplacement relatif.

Signal	Broche proposée	Fonction
A	D2	Entrée d'interruption INT1
B	D3	Entrée numérique permettant de déterminer le sens

4.2 Lecture du sens de rotation

L'interruption est déclenchée sur une transition du signal A. La valeur du signal B est alors lue afin de déterminer le sens. Le sens exact dépend du câblage et de l'encodeur : il devra être vérifié expérimentalement.

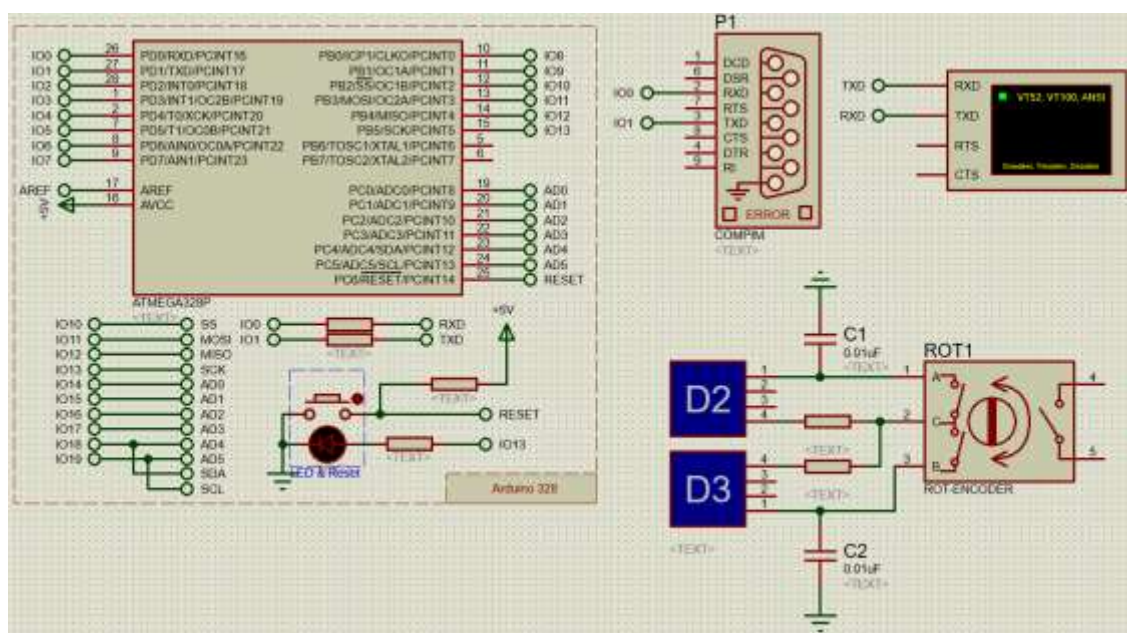


4.3 Contextes d'application de l'encodeur incrémental

N°	Contexte d'application	Utilisation de l'encodeur	Grandeur mesurée / déterminée
1	Position d'un moteur	Comptage des impulsions lors de la rotation de l'axe	Position angulaire
2	Vitesse d'un moteur	Comptage du nombre d'impulsions pendant une durée donnée	Vitesse de rotation
3	Sens de rotation d'un moteur	Comparaison du déphasage entre les voies A et B	Sens horaire / antihoraire
4	Ascenseur	Mesure de la rotation de la poulie	Position de la cabine
5	Convoyeur industriel	Mesure de la rotation d'un rouleau	Déplacement / vitesse

N°	Contexte d'application	Utilisation de l'encodeur	Grandeur mesurée / déterminée
6	Robot mobile	Encodeur placé sur les roues	Distance parcourue / vitesse
7	Machine CNC	Mesure de la rotation des axes	Position de l'outil
8	Imprimante 3D	Contrôle du déplacement des axes	Position de la tête / du plateau
9	Volet ou portail motorisé	Comptage des rotations du moteur	Position d'ouverture
10	Molette de réglage	Détection des impulsions lors de la rotation	Réglage d'une consigne
11	Roue codeuse	Comptage des impulsions générées par une roue	Distance parcourue
12	Bras robotisé	Mesure de la rotation des articulations	Position angulaire
13	Tachymètre	Comptage des impulsions sur une période	Vitesse de rotation
14	Commande de volume / luminosité	Utilisation d'un encodeur rotatif par l'utilisateur	Valeur de consigne
15	Système de positionnement	Comptage et décomptage des impulsions	Position relative

Schéma structurel



Programme à compléter, programme et commentaire

```

/*
 * PPE - Codeur incrémental
 * -----
 * Objectif :
 *   - Détecter les changements d'état du signal A
 *   - Lire le signal B pour déterminer le sens de rotation
 *   - Compter les impulsions dans la variable position
 *   - Afficher la position sur le moniteur série
 *
 * Le codeur possède deux sorties :
 *   - voie A : signal utilisé pour déclencher l'interruption
 *   - voie B : signal utilisé pour déterminer le sens de rotation
 */

// Broche connectée à la voie A du codeur
const int encoderA = _____;

// Broche connectée à la voie B du codeur
const int encoderB = _____;

// Variable contenant la position du codeur
// Elle est modifiée dans la routine d'interruption
volatile long position = 0;

/*
 * oldPosition mémorise la dernière position
 * qui a été affichée.
 *
 * static permet de conserver sa valeur entre
 * deux exécutions de loop().
 */
static long oldPosition = _____;

/*
 * Routine d'interruption appelée lors d'un changement
 * d'état de la voie A.
 */
void encoderISR()
{
    /*
     * On lit l'état de la voie B.
     *
     * La combinaison des voies A et B permet de déterminer
     * le sens de rotation.
     *
     * Si B est à l'état HAUT :
     */
    _____

```

```

* | À COMPLÉTER
*
* Sinon :
*
*/
if (digitalRead(encoderB) == HIGH)

    position++;          // À COMPLÉTER : que doit-on faire ?

else

    position--;          // À COMPLÉTER : que doit-on faire ?
}

/*
 * Initialisation du programme
 */
void setup()
{
    // Configure la voie A comme entrée avec résistance
    // de rappel interne.
    pinMode(encoderA, _____);

    // Configure la voie B comme entrée avec résistance
    // de rappel interne.
    pinMode(encoderB, _____);

    /*
     * Mise en place de l'interruption.
     *
     * L'interruption doit être déclenchée :
     * 1. sur la broche correspondant à encoderA
     * 2. lorsque l'état de cette broche change (front montant en 2)
     * 3. en appelant la fonction encoderISR()
     */
    attachInterrupt(
        _____,
        _____,
        _____
    );

    // Initialisation de la communication série
    // à 115200 bauds.
    Serial.begin(_____);
}

```

```

/*
 * Boucle principale
 */
void loop()
{

    /*
     * On vérifie si la position du codeur a changé
     * depuis la dernière mesure.
     */
    if (position _____ oldPosition)
    {
        // On mémorise la nouvelle position.
        oldPosition = _____;

        // Affichage d'un texte sur le moniteur série.
        Serial.print("Position = ");

        // Affichage de la position du codeur.
        Serial.println(_____);
    }
}

```

4.4 Travail pratique n°2 – Encodeur

- Réaliser le câblage de l'encodeur : voie A sur D3 et voie B sur D4.
- Compiler et téléverser le programme fourni.
- Tourner lentement l'encodeur dans un sens puis dans l'autre.
- Observer l'évolution de la variable position.
- Vérifier expérimentalement quel signe correspond à chaque sens.
- Modifier le programme pour inverser le sens si nécessaire.
- Ajouter une limitation de position entre -10 et +10.
- Ajouter un message dans le Moniteur Série lorsque la position atteint une limite.

4.5 Mise en commun : interruption et interface utilisateur

L'encodeur est particulièrement intéressant pour réaliser une interface homme-machine. La rotation peut modifier une valeur et un bouton intégré ou externe peut valider un choix. Cette logique sera réutilisée dans le projet final pour naviguer dans des menus.

Questionnaire

1. Pourquoi utilise-t-on deux signaux A et B ?
2. Comment déterminer le sens de rotation ?

3. Quel est le rôle de la variable position ?
4. Pourquoi position est-elle déclarée volatile ?
5. Quelle différence existe-t-il entre une position absolue et une position incrémentale ?
6. Quel problème apparaît si l'encodeur est tourné très rapidement ?

Partie activités durée conseillée : 30min

5 Bibliothèques Arduino : installer, explorer, réutiliser

5.1 Pourquoi utiliser une bibliothèque ?

Une bibliothèque permet de regrouper des fonctions et des classes réutilisables. Dans un système embarqué, elle peut prendre en charge une partie de la communication avec un composant et fournir au programmeur une interface plus simple.

L'objectif de cette partie n'est pas de recopier un programme trouvé sur Internet. L'étudiant doit apprendre à identifier une bibliothèque adaptée, l'installer, consulter ses exemples, comprendre les fonctions principales puis adapter l'exemple au cahier des charges.

5.2 Méthode de travail

- Identifier le composant à utiliser.
- Rechercher son fabricant et sa documentation.
- Identifier le nom de la bibliothèque recommandée.
- Installer la bibliothèque avec le gestionnaire de bibliothèques Arduino.
- Ouvrir les exemples fournis par la bibliothèque.
- Identifier les fonctions d'initialisation, de lecture et d'affichage.
- Tester l'exemple sans modification lorsque cela est possible.
- Adapter progressivement l'exemple au câblage et au cahier des charges.
- Conserver dans le compte rendu le nom des bibliothèques utilisées.

Composant	Bibliothèque à rechercher	Dépendance / remarque
LCD RGB Grove	Grove LCD RGB Backlight / bibliothèque Seeed	Exemples LCD fournis avec la bibliothèque
DS18B20	DallasTemperature	Nécessite également OneWire
DHT11	DHT sensor library by Adafruit	Nécessite Adafruit Unified Sensor

LCD RGB Grove	DS18B20	DHT11
		

5.3 Travail pratique n°3 – LCD RGB Grove

Le LCD RGB Grove communique en I²C. La bibliothèque fournie par Seeed permet notamment d'initialiser l'afficheur, de positionner le curseur, d'afficher du texte et de modifier la couleur du rétroéclairage. La documentation du constructeur fournit plusieurs exemples tels que HelloWorld, SerialDisplay, setColor et setCursor.

Travail demandé

- 1- Connecter le LCD RGB Grove sur le bus I²C de la carte Grove.
- 2- Ouvrir le gestionnaire de bibliothèques de l'IDE Arduino.
- 3- Rechercher la bibliothèque associée au LCD RGB Grove.
- 4- Identifier son auteur et l'installer.
- 5- Rechercher les exemples associés à cette bibliothèque.
- 6- Ouvrir l'exemple HelloWorld et identifier les éléments spécifiques à la bibliothèque.
- 7- Compiler et téléverser l'exemple.
- 8- Modifier le texte affiché.
- 9- Modifier la couleur du rétroéclairage.
- 10- Utiliser setCursor() pour afficher une information sur la deuxième ligne.
- 11- Afficher une valeur issue de l'encodeur.

5.4 Questions sur les bibliothèques

1. Quel fichier d'en-tête doit être inclus pour utiliser la bibliothèque du LCD ?
2. Quel objet permet de piloter l'afficheur ?
3. Quelle fonction initialise l'afficheur ?
4. Quelle fonction permet de modifier la couleur du rétroéclairage ?
5. Pourquoi l'exemple fourni par le constructeur constitue-t-il un bon point de départ ?
6. Quelle différence existe-t-il entre le code de l'application et le code fourni par la bibliothèque ?

Partie activités durée conseillée : 30min

6 Acquisition de capteurs avec des bibliothèques

6.1 Objectif

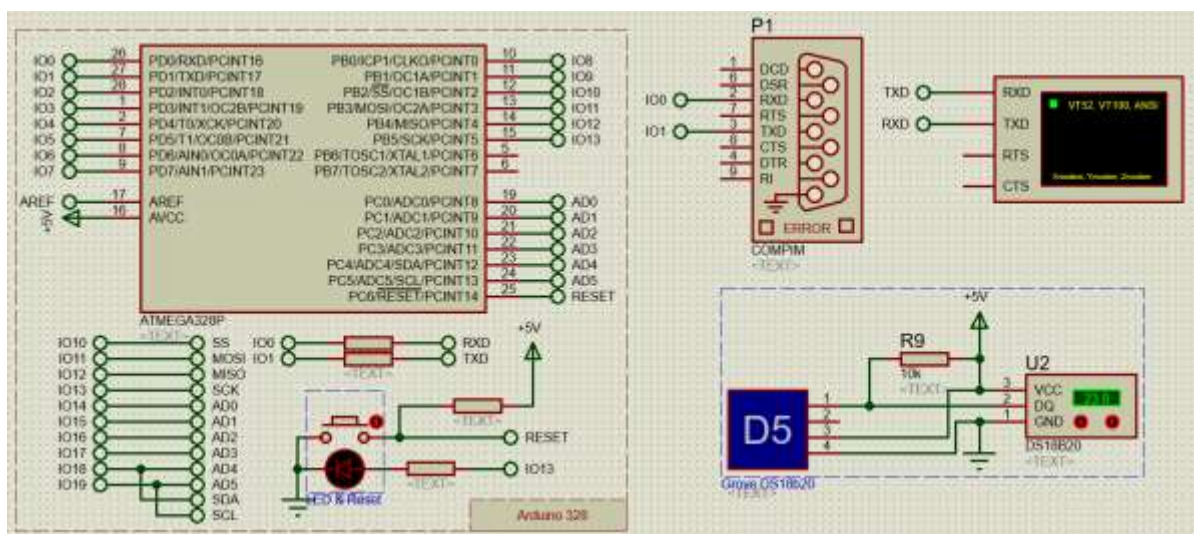
Cette partie est volontairement moins guidée. À partir d'un cahier des charges, vous devez rechercher les bibliothèques adaptées, les installer, étudier leurs exemples puis construire votre propre programme.

6.2 Capteur DS18B20

Le DS18B20 utilise un bus 1-Wire. La bibliothèque DallasTemperature simplifie l'accès aux fonctions de mesure. Elle s'appuie sur la bibliothèque OneWire.

Point matériel : Le bus 1-Wire nécessite une résistance de rappel entre la ligne de données et l'alimentation selon le montage du capteur. Vérifier le module Grove utilisé.

Schéma structurel



6.3 Mission autonomie A – DS18B20

À partir du cahier des charges suivant, développer votre programme.

Le système doit mesurer la température à l'aide du DS18B20.

La température doit être affichée dans le Moniteur Série.

La mesure doit être réalisée périodiquement.

Le programme doit utiliser une bibliothèque dédiée au DS18B20.

Le nom de la bibliothèque et les fonctions principales utilisées doivent être indiqués dans le compte rendu.

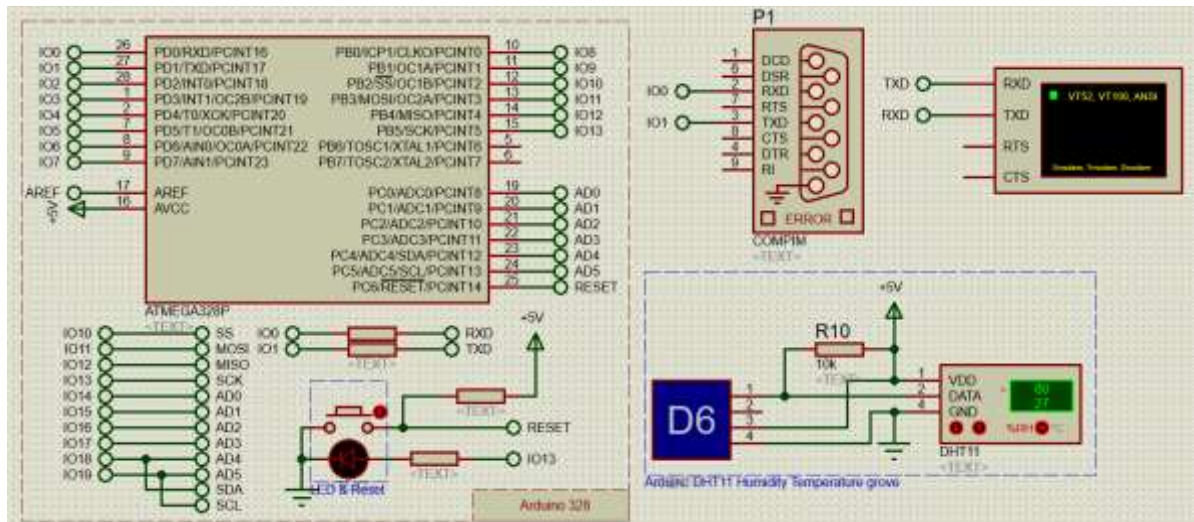
Un exemple fourni avec la bibliothèque doit être identifié et étudié avant le développement.

6.4 Capteur DHT11

Le DHT11 fournit une température et une humidité relative. Une bibliothèque dédiée permet de gérer le protocole de communication et de convertir les données reçues en valeurs directement exploitables.

La bibliothèque DHT d'Adafruit utilise également Adafruit Unified Sensor. Cette dépendance doit être installée si l'IDE la demande.

Schéma structurel



6.5 Mission autonomie B – DHT11

Identifier la bibliothèque DHT adaptée au DHT11.

Installer la bibliothèque et sa dépendance éventuelle.

Identifier au moins un exemple fourni.

Tester l'exemple.

Adapter le programme afin d'afficher température et humidité dans le Moniteur Série.

Choisir une période de mesure cohérente avec le fonctionnement du capteur.

Identifier les valeurs obtenues et leurs unités.

6.6 Format JSON imposé

Les données devront être transmises sous forme de JSON afin de disposer d'une représentation structurée et facilement exploitable par un autre système informatique.

Le format imposé pour ce TP est le suivant :

```
{"temperature":23.5,"humidite":48.0}
```

Lorsque les deux capteurs sont disponibles, le message complet devra respecter le format suivant :

```
{"temperature":23.5,"humidite":48.0,"ds18b20":23.6}
```

Les noms des champs sont imposés. Ne pas remplacer temperature, humidite ou ds18b20 par d'autres noms.

6.7 Mission autonomie C – Acquisition et JSON

Acquérir la température du DS18B20.

Acquérir la température et l'humidité du DHT11.

Construire le message JSON demandé.

Envoyer un message JSON périodiquement sur la liaison série.

Vérifier que chaque message commence par { et se termine par }.

Vérifier que les champs sont séparés par des virgules et que les noms sont entre guillemets.

Observer le résultat dans le Moniteur Série.

Justifier dans le compte rendu la structure du JSON produit.

6.8 Exemple de résultat attendu

```
{"temperature":22.8,"humidite":51.0,"ds18b20":22.9}
```

```
{"temperature":22.9,"humidite":51.0,"ds18b20":23.0}
```

```
{"temperature":23.0,"humidite":50.0,"ds18b20":23.1}
```

Remarque : Les valeurs numériques ci-dessus sont des exemples de format et ne constituent pas des valeurs de mesure à reproduire.

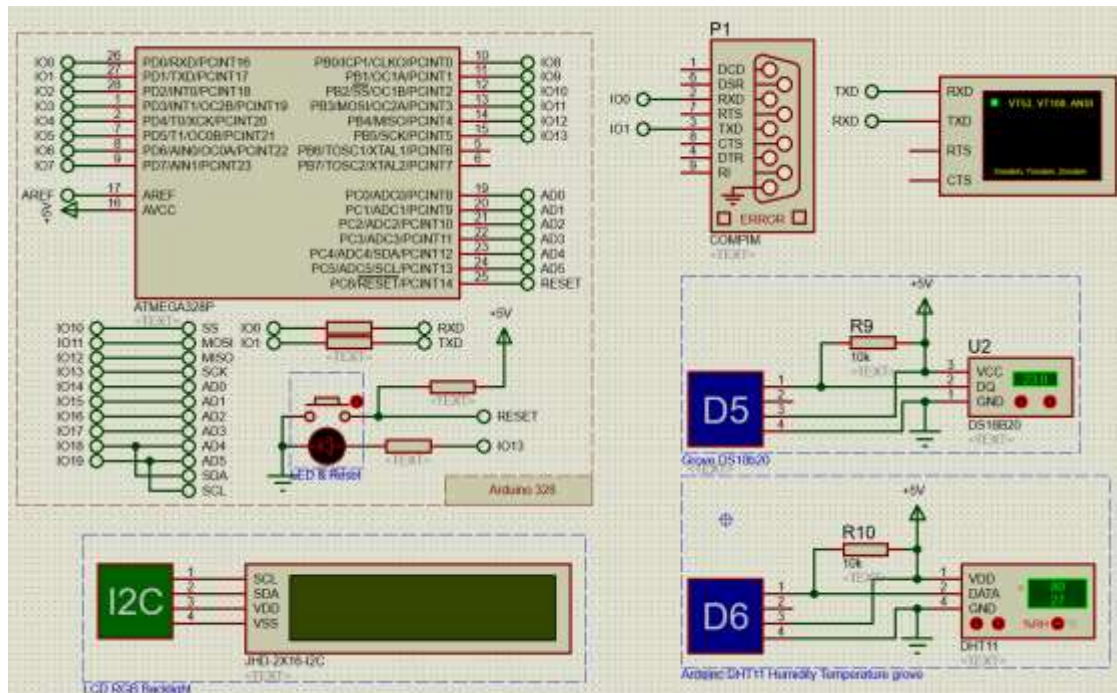
Partie activités durée conseillée : 55min

7 Affichage des mesures sur le LCD RGB

7.1 Cahier des charges

Le système doit afficher les informations de mesure sur le LCD RGB tout en continuant à transmettre le JSON sur la liaison série.

Élément	Exigence
LCD	Affichage sur deux lignes
Ligne 1	Température en °C
Ligne 2	Humidité en %
Liaison série	Transmission du JSON imposé
Mise à jour	Périodique
Encodeur	Préparé pour la navigation dans le projet final



Travail demandé

- Reprendre le programme validé avec les capteurs.
- Ajouter l'utilisation de la bibliothèque LCD RGB.
- Initialiser l'afficheur dans setup().
- Afficher la température sur la première ligne.
- Afficher l'humidité sur la deuxième ligne.
- Effacer ou réécrire correctement les anciennes valeurs afin d'éviter les caractères résiduels.
- Conserver simultanément l'émission du JSON sur le Moniteur Série.
- Choisir une couleur de rétroéclairage cohérente avec le fonctionnement du système.

7.2 Analyse

1. Pourquoi est-il préférable de séparer l'acquisition, le traitement et l'affichage ?
2. Pourquoi faut-il éviter d'actualiser inutilement l'afficheur à chaque passage de loop() ?
3. Quelle partie du programme dépend directement de la bibliothèque LCD ?
4. Quelle partie du programme pourrait être réutilisée si l'afficheur était remplacé ?
5. Quel avantage présente le format JSON pour transmettre les mesures à une autre application ?

8 Projet de synthèse – Gestion d'un menu d'affichage

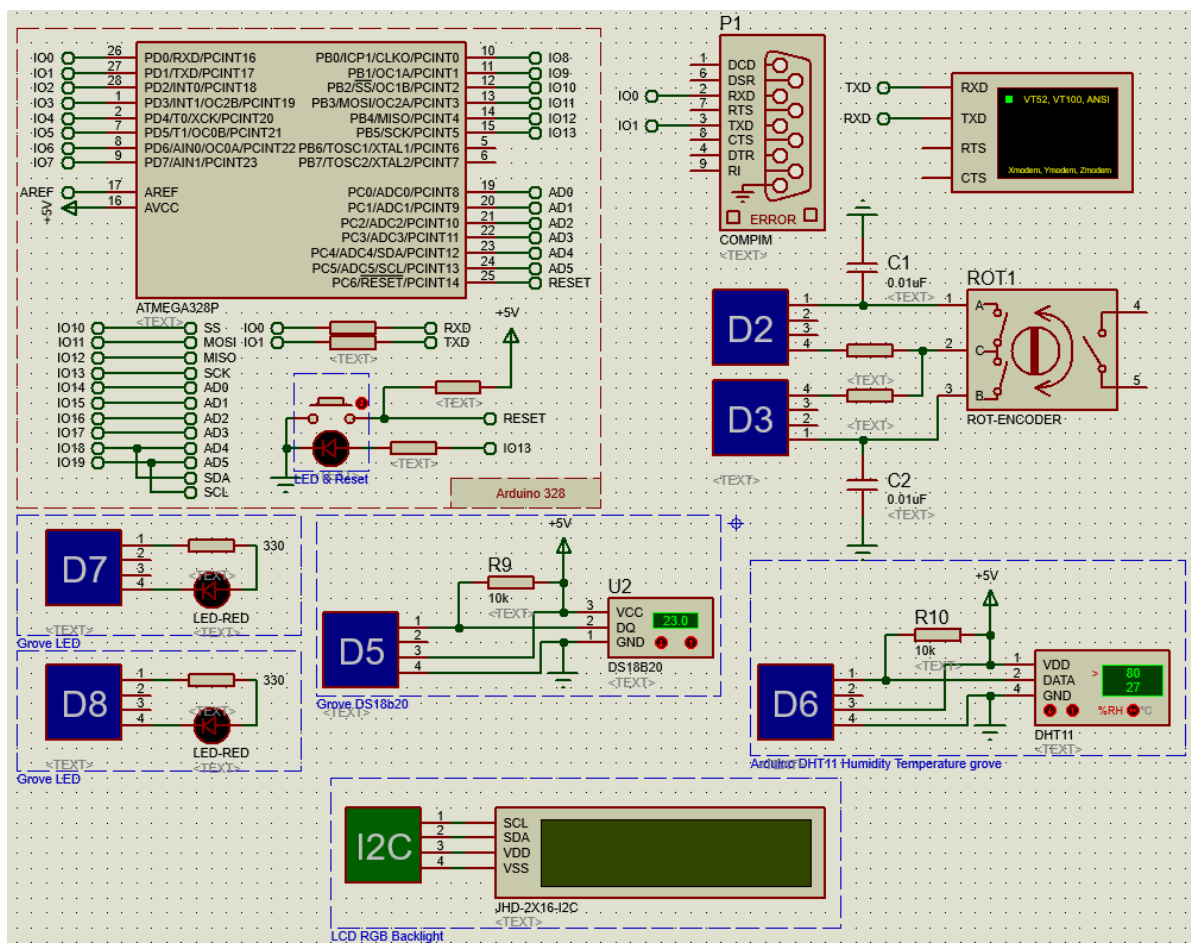
8.1 Contexte

Vous devez réaliser l'interface utilisateur d'un petit système de surveillance environnementale. Le système mesure la température et l'humidité et les affiche sur un LCD RGB. Un opérateur doit pouvoir naviguer dans plusieurs écrans à l'aide d'un encodeur incrémental.

8.2 Cahier des charges

Fonction	Cahier des charges
Mise sous tension	Initialiser les entrées, les capteurs, l'afficheur et la liaison série.
Menu 0	Afficher un écran d'accueil / état du système.
Menu 1	Afficher la température du DHT11 et du DS18B20.
Menu 2	Afficher l'humidité du DHT11.
Menu 3	Afficher la position de l'encodeur.
Rotation	La rotation de l'encodeur change le menu sélectionné.
Validation	L'appui sur le bouton permet de sélectionner ou valider un menu.
Série	Transmettre périodiquement les mesures au format JSON imposé.
LCD	Actualiser uniquement les informations nécessaires.
Réactivité	La détection de l'encodeur doit utiliser les interruptions.
Code	Utiliser des fonctions distinctes et des noms de variables explicites.

Schéma structurel



8.3 Organisation logicielle conseillée

L'application devra être structurée afin d'éviter de concentrer tout le traitement dans `loop()`. Vous pouvez par exemple prévoir des fonctions dédiées à l'acquisition, à l'affichage, au menu et à la transmission.

Fonction possible	Rôle
<code>lireCapteurs()</code>	Mettre à jour les mesures
<code>afficherMenu()</code>	Afficher le menu courant
<code>afficherMesures()</code>	Afficher les valeurs sur le LCD
<code>envoyerJSON()</code>	Construire et transmettre le JSON
<code>gererMenu()</code>	Traiter les événements utilisateur

encoderISR()	Mettre à jour la position de l'encodeur
boutonISR()	Signaler un appui

8.4 Contraintes de programmation

- Utiliser une interruption pour l'encodeur.
- Utiliser une interruption pour le bouton si le montage retenu permet sa validation sur D2.
- Ne pas réaliser d'affichage LCD dans une ISR.
- Ne pas utiliser delay() dans une ISR.
- Utiliser les bibliothèques nécessaires aux capteurs et à l'afficheur.
- Identifier les bibliothèques et leurs auteurs dans le compte rendu.
- Conserver le format JSON imposé.
- Structurer le programme avec plusieurs fonctions.
- Utiliser des variables explicites et des commentaires utiles.
- Tester chaque sous-fonction avant l'intégration finale.

8.5 Démarche de développement du projet

8.5.1 Étape 1 – Valider les capteurs

1. Tester séparément le DS18B20.
2. Tester séparément le DHT11.
3. Vérifier les valeurs dans le Moniteur Série.
4. Identifier les bibliothèques et les exemples utilisés.

8.5.2 Étape 2 – Valider l'affichage

5. Tester le LCD avec un exemple constructeur.
6. Afficher une valeur fixe.
7. Afficher une mesure réelle.
8. Vérifier l'effacement et la réécriture des lignes.

8.5.3 Étape 3 – Valider l'encodeur

9. Vérifier les deux voies A et B.
10. Vérifier le sens de rotation.
11. Vérifier la modification de la variable de position.
12. Vérifier l'interruption.
13. Tester le bouton de validation.

8.5.4 Étape 4 – Construire le menu

14. Créer une variable représentant le numéro du menu.
15. Faire varier cette variable avec l'encodeur.
16. Limiter la valeur à l'intervalle des menus.
17. Afficher le menu courant.
18. Utiliser le bouton pour valider l'écran.
19. Retourner au menu principal selon une logique définie.

8.5.5 Étape 5 – Ajouter le JSON

20. Construire le message à partir des dernières mesures disponibles.
21. Transmettre périodiquement le message.
22. Vérifier le format avec plusieurs mesures.
23. Vérifier que l'affichage LCD reste fonctionnel pendant la transmission.

Test	Résultat attendu	Validation
Rotation horaire	Le menu avance	☐
Rotation antihoraire	Le menu recule	☐
Appui bouton	Le menu est sélectionné	☐
DS18B20	Température correcte	☐
DHT11	Température + humidité	☐
LCD	Informations lisibles	☐
Série	JSON valide et périodique	☐
Redémarrage	Initialisation correcte	☐

8.6 Critères de réussite

- Le programme compile sans erreur.
- Les interruptions sont réellement utilisées pour les événements demandés.
- L'encodeur permet une navigation fiable.
- Les capteurs fournissent des valeurs cohérentes.
- Le LCD affiche les informations demandées.
- Le JSON respecte exactement le format imposé.
- Le programme est structuré et lisible.
- Les bibliothèques utilisées sont identifiées.
- Le système reste réactif pendant son fonctionnement.

8.7 Questionnaire de synthèse

1. Quelle différence existe-t-il entre une scrutation et une interruption ?
2. Quelles sont les deux broches d'interruption externe de l'Arduino Uno ?
3. Quel est le rôle de volatile ?
4. Pourquoi une ISR doit-elle être courte ?
5. Comment deux signaux permettent-ils à un encodeur incrémental de déterminer un sens de rotation ?
6. Quel est le rôle d'une bibliothèque Arduino ?
7. Pourquoi étudier un exemple fourni avec une bibliothèque avant de développer son programme ?
8. Quelles bibliothèques sont utilisées pour le DS18B20 et le DHT11 ?
9. Pourquoi le DS18B20 utilise-t-il la bibliothèque OneWire ?
10. Pourquoi le DHT11 nécessite-t-il une bibliothèque adaptée à son protocole ?
11. Quel est l'intérêt du format JSON dans ce système ?
12. Pourquoi ne doit-on pas mettre l'affichage LCD dans une routine d'interruption ?
13. Quel rôle joue l'encodeur dans l'interface homme-machine ?
14. Comment le programme peut-il éviter de réactualiser inutilement le LCD ?
15. Identifier les blocs Acquérir → Traiter → Communiquer → Interagir dans le projet.

9 Conclusion du TP

Au cours de ce TP, vous avez fait évoluer votre manière de programmer un système embarqué. Après avoir utilisé la scrutation dans le TP1, vous avez découvert la programmation événementielle grâce aux interruptions. Vous avez également appris à exploiter des composants complexes à l'aide de bibliothèques logicielles.

La démarche acquise est réutilisable pour de nombreux systèmes embarqués : identifier un composant, rechercher sa documentation, installer sa bibliothèque, étudier les exemples, tester séparément le matériel, puis intégrer progressivement les fonctions dans une application structurée.

Le projet final combine les principales notions du TP : interruptions, encodeur, bibliothèque, capteurs, affichage, liaison série et données structurées. Il constitue une transition vers des applications plus complexes et vers la programmation de systèmes connectés qui seront étudiés dans la suite de la formation.

Annexe A – Trace de recherche des bibliothèques

Composant	Bibliothèque trouvée	Auteur	Version	Exemple étudié
LCD RGB				
DS18B20				
OneWire				
DHT11				
Adafruit Unified Sensor				

Tableau de synthèse — Interruptions externes avec Arduino Uno

Sur **Arduino Uno**, les interruptions externes matérielles principales sont associées aux broches :

- **D2** → interruption **INT0**
- **D3** → interruption **INT1**

API / fonction	Rôle	Syntaxe générique	Exemple
<code>attachInterrupt()</code>	Associer une fonction à une interruption externe	<code>attachInterrupt(broche, ISR, mode);</code>	<code>attachInterrupt(26, ISR_bouton, RISING);</code>
<code>detachInterrupt()</code>	Désactiver une interruption	<code>detachInterrupt(broche);</code>	<code>detachInterrupt(26);</code>
<code>digitalPinToInterrupt()</code>	Obtenir l'interruption associée à une broche	<code>digitalPinToInterrupt(broche)</code>	<code>digitalPinToInterrupt(26)</code>
<code>RISING</code>	Déclenchement sur front montant	<code>..., RISING);</code>	<code>attachInterrupt(26, ISR, RISING);</code>
<code>FALLING</code>	Déclenchement sur	<code>..., FALLING);</code>	<code>attachInterrupt(26, ISR, FALLING);</code>

API / fonction	Rôle	Syntaxe générique	Exemple
	front descendant		
CHANGE	Déclenche ment sur changement d'état	..., CHANGE);	attachInterrupt (26, ISR, CHANGE);
LOW	Déclenche ment lorsque l'entrée est à l'état bas	..., LOW);	attachInterrupt (26, ISR, LOW);
HIGH	Déclenche ment lorsque l'entrée est à l'état haut	..., HIGH);	attachInterrupt (26, ISR, HIGH);
volatile	Variable partagée avec la fonction d'interruption	volatile type variable;	volatile bool etat = false;

Exemple — Bouton sur D2

```
// Broche utilisée pour le bouton poussoir.
// Sur Arduino Uno, la broche D2 possède une interruption externe INT0.
const int bouton = 2;

// Variable globale modifiée par la routine d'interruption.
// volatile indique au compilateur que sa valeur peut être modifiée
// à tout moment par une interruption.
volatile int compteur = 0;

// =====
// ROUTINE D'INTERRUPTION
// =====

// Cette fonction est exécutée automatiquement lorsqu'une
// interruption est détectée sur la broche du bouton.
void ISR_bouton() {

    // Incrémente le compteur à chaque appui détecté.
    compteur++;
}
```

```
// =====  
// INITIALISATION  
// =====  
  
void setup() {  
  
    // Configure la broche du bouton en entrée avec résistance  
    // de rappel interne vers VCC.  
    //  
    // Au repos : bouton = HIGH  
    // Bouton appuyé : bouton = LOW  
    pinMode(bouton, INPUT_PULLUP);  
  
    // Association de la broche à une interruption externe.  
    attachInterrupt(  
  
        // Convertit le numéro de broche Arduino (2)  
        // en numéro d'interruption (INT0 sur Arduino Uno).  
        digitalPinToInterrupt(bouton),  
  
        // Fonction qui sera exécutée lors de l'interruption.  
        ISR_bouton,  
  
        // Déclenche l'interruption lors d'un front descendant :  
        // HIGH -> LOW  
        FALLING  
  
    );  
  
    // Initialise la communication série à 115200 bauds.  
    Serial.begin(115200);  
}  
  
// =====  
// PROGRAMME PRINCIPAL  
// =====  
  
void loop() {  
  
    // Affiche la valeur actuelle du compteur.  
    Serial.println(compteur);  
  
    // Attend 500 ms avant la prochaine affichage.
```

```
//  
// IMPORTANT :  
// même pendant ce delay(), l'interruption reste active.  
// Un appui sur le bouton peut donc modifier compteur.  
delay(500);  
}
```

9.1 Le format JSON

9.1.1 Pourquoi utiliser JSON ?

Une liaison série permet à deux systèmes électroniques ou informatiques d'échanger des données.

Par exemple, une carte Arduino peut mesurer :

- la température ;
- l'humidité ;
- la vitesse d'un moteur ;
- la position d'un capteur ;
- l'état d'une LED.

Elle peut ensuite transmettre ces informations vers un ordinateur ou une autre carte.

Une première solution serait d'envoyer uniquement les valeurs :

```
23.5;48.2;1250;1
```

Mais cette information est difficile à interpréter : que représente chaque valeur ?

Pour rendre les données **structurées, identifiables et facilement exploitables par un programme**, on peut utiliser le format **JSON**.

La même information peut alors être transmise sous la forme :

```
{  
  "temperature": 23.5,  
  "humidite": 48.2,  
  "vitesse": 1250,  
  "led": 1  
}
```

Le récepteur sait alors immédiatement à quoi correspond chaque donnée.

9.1.2 Qu'est-ce que JSON ?

JSON signifie :

JavaScript Object Notation

Il s'agit d'un **format texte** permettant de représenter et d'échanger des données structurées.

Bien que son nom fasse référence à JavaScript, JSON est aujourd'hui utilisé par de très nombreux langages de programmation :

- C / C++
- Python
- Java
- JavaScript
- C#
- PHP
- etc.

JSON est particulièrement utilisé dans les systèmes communicants :

- objets connectés (IoT) ;
- API Web ;
- applications mobiles ;
- serveurs ;
- systèmes embarqués ;
- communications entre équipements.

Dans notre TP Arduino, JSON est utilisé pour **structurer les informations transmises sur la liaison série**.

9.1.3 Structure générale d'un objet JSON

Un objet JSON est délimité par des **accolades** :

```
{  
  ...  
}
```

À l'intérieur, les données sont organisées sous la forme :

"clé" : valeur

Par exemple :

```
{  
  "temperature": 23.5  
}
```

Ici :

- "temperature" est la **clé** ;
- 23.5 est la **valeur**.

On peut voir cette association comme :

clé → valeur
"temperature" → 23.5

La clé permet donc d'identifier précisément la donnée.

9.1.4 Les différents types de données

JSON permet principalement de représenter plusieurs types de données.

9.1.4.1 Nombre

```
{  
  "temperature": 23.5,  
  "vitesse": 1250  
}
```

Les nombres ne sont **pas placés entre guillemets**.

9.1.4.2 Texte

Une chaîne de caractères est placée entre guillemets :

```
{  
  "nom": "Arduino",  
  "etat": "OK"  
}
```

Attention à la différence :

```
"vitesse": 1250
```

signifie que 1250 est un nombre.

Alors que :

```
"vitesse": "1250"
```

signifie que 1250 est du texte.

9.1.4.3 Booléen

JSON utilise les valeurs :

```
true  
false
```

Exemple :

```
{  
  "moteur": true,  
  "defaut": false  
}
```

9.1.4.4 Tableau

Un tableau est délimité par des crochets :

```
{  
  "temperatures": [21.5, 22.1, 23.0, 23.5]  
}
```

Les éléments sont séparés par des virgules.

9.1.4.5 Exemple appliqué à Arduino

Imaginons une carte Arduino équipée :

- d'un capteur de température ;
- d'un capteur d'humidité ;
- d'un moteur ;
- d'une LED indiquant l'état du système.

Arduino peut transmettre périodiquement :

```
{  
  "temperature": 24.3,  
  "humidite": 52.7,  
  "vitesse": 850,  
  "moteur": true,  
  "defaut": false  
}
```

Le message contient plusieurs informations.

Clé	Valeur	Signification
temperature	24.3	température mesurée en °C
humidite	52.7	humidité mesurée en %
vitesse	850	vitesse du moteur
moteur	true	moteur en fonctionnement
defaut	false	aucun défaut

Le programme côté réception peut donc rechercher la donnée dont il a besoin.

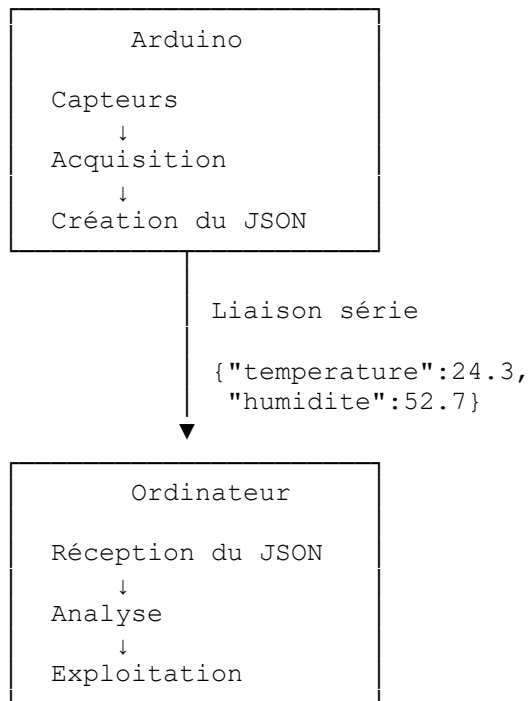
9.1.4.6 JSON et liaison série

Il est important de comprendre que **JSON n'est pas un moyen de communication**.

La liaison série assure le transport des caractères.

JSON assure la **structuration des informations transportées**.

On peut représenter le fonctionnement ainsi :



La liaison série transporte donc une chaîne de caractères telle que :

```
{"temperature":24.3,"humidite":52.7}
```

9.1.4.7 Les règles essentielles de JSON

Pour qu'un message soit correctement interprété, il faut respecter certaines règles.

9.1.4.8 Les clés sont entre guillemets

Correct :

```
{
  "temperature": 23.5
}
```

Incorrect :

```
{
  temperature: 23.5
}
```

9.1.4.9 Les éléments sont séparés par des virgules

Correct :

```
{
  "temperature": 23.5,
  "humidite": 50
}
```

Incorrect :

```
{
  "temperature": 23.5
  "humidite": 50
}
```

9.1.4.10 Les accolades délimitent un objet

```
{
  "temperature": 23.5
}
```

9.1.4.11 Les crochets délimitent un tableau

```
{
  "mesures": [20, 21, 22, 23]
}
```

9.1.4.12 Les chaînes de caractères utilisent des guillemets doubles

Correct :

```
{
  "etat": "OK"
}
```

9.1.5 Exemple : transmission Arduino

Arduino peut construire un message JSON puis l'envoyer sur le port série.

Le message transmis peut être :

```
{"temperature":23.5,"humidite":48.2,"led":true}
```

Par exemple :

```
Serial.print("{");
Serial.print("\"temperature\":");
Serial.print(temperature);
Serial.print(",");
Serial.print("\"humidite\":");
Serial.print(humidite);
Serial.print(",");
Serial.print("\"led\":");
Serial.print(led);
Serial.println("}");
```

Le moniteur série reçoit alors :

```
{"temperature":23.5,"humidite":48.2,"led":true}
```

Cependant, cette méthode devient rapidement difficile à maintenir lorsque le nombre de données augmente.

On utilise donc généralement une **bibliothèque JSON**.

Une bibliothèque permet de simplifier la création et l'analyse des messages JSON.

Par exemple, avec une bibliothèque telle que **ArduinoJson**, on peut construire un objet JSON de manière plus lisible :

```
JsonDocument doc;

doc["temperature"] = temperature;
doc["humidite"] = humidite;
doc["led"] = true;

serializeJson(doc, Serial);
Serial.println();
```

Le résultat transmis sur la liaison série sera par exemple :

```
{"temperature":23.5,"humidite":48.2,"led":true}
```

La bibliothèque prend en charge la syntaxe JSON.

Le programmeur se concentre donc davantage sur les **données à transmettre**.

9.3 Pourquoi utiliser JSON ?

L'utilisation de JSON présente plusieurs avantages.

9.3.1 Lisibilité

Un message :

```
{
  "temperature": 23.5,
  "humidite": 48.2
}
```

est facilement compréhensible par un humain.

9.3.2 Identification des données

Chaque valeur possède une clé :

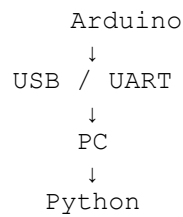
```
"temperature"
"humidite"
```

Il n'est donc pas nécessaire de connaître l'ordre des valeurs pour comprendre le message.

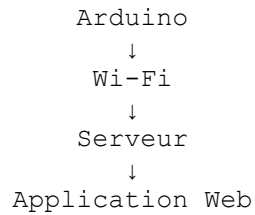
9.3.3 Interopérabilité

De nombreux langages et systèmes informatiques savent lire et produire du JSON.

Arduino peut donc communiquer avec :



ou :



9.3.4 Évolutivité

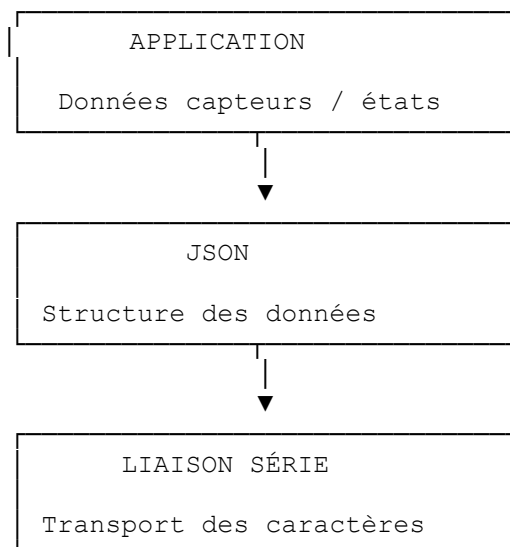
Il est possible d'ajouter une nouvelle donnée sans modifier nécessairement toute la structure :

```

{
  "temperature": 23.5,
  "humidite": 48.2,
  "pression": 1013.2
}
  
```

9.3.5 JSON dans un système communicant

Dans un système embarqué, on peut donc distinguer trois niveaux :



9.4 À retenir :

La liaison série transporte les données, tandis que JSON définit une structure permettant de les organiser et de les interpréter.

9.5 Exemple de message complet

Dans notre application Arduino, un message peut être :

```
{  
  "id": 12,  
  "temperature": 24.6,  
  "vitesse": 850,  
  "moteur": true,  
  "etat": "OK"  
}
```

On peut alors interpréter ce message comme :

ID du système	→ 12
Température	→ 24,6 °C
Vitesse moteur	→ 850
Moteur	→ en fonctionnement
État du système	→ OK

Le même message peut être envoyé par Arduino et reçu par un programme Python, une application Web ou une autre carte électronique.

9.6 À retenir

JSON est un format texte permettant de représenter et d'échanger des données structurées.

Sa structure repose notamment sur :

objet	→ { }
tableau	→ []
clé	→ "nom"
valeur	→ nombre, texte, true, false...
séparation	→ ,

Exemple :

```
{  
  "temperature": 24.6,  
  "moteur": true,  
  "etat": "OK"  
}
```

Dans notre TP :

Arduino produit les données → JSON les structure → la liaison série les transporte → le système récepteur les exploite.



API arduinoJson

API / fonction	Rôle	Syntaxe générique	Exemple
JsonDocument	Créer un document JSON	JsonDocument nom;	JsonDocument doc;
doc["clé"] = valeur	Ajouter une donnée au JSON	document["clé"] = valeur;	doc["id"] = 12;
serializeJson()	Émettre le JSON	serializeJson(doc, ent, destination);	serializeJson(doc, Serial);
Serial.available()	Vérifier si un message est reçu	Serial.available()	if (Serial.available())
Serial.readStringUntil()	Lire le message reçu	Serial.readStringUntil(caractère);	messageRecu = Serial.readStringUntil('\n');
deserializeJson()	Analyser le message JSON reçu	deserializeJson(doc, document, texte);	deserializeJson(doc, messageRecu);
DeserializationError	Vérifier si le JSON reçu est valide	if (erreur)	if (erreur) { ... }
doc["clé"]	Lire une donnée du JSON	variable = document["clé"];	temperature = doc["temperature"];

